



Engenharia de Computação
Inteligência Artificial
06 - Lógica de Primeira Ordem
Parte II - Inferência em LPO

Organização da aula

- Retomada: linguagem, quantificadores, fatos, regras e consultas em LPO.
- Instanciação universal e existencial.
- Substituição, modus ponens generalizado e unificação.
- Cláusulas definidas e encadeamento direto (*forward chaining*).
- Encadeamento reverso (*backward chaining*) e retrocesso (*backtracking*).
- Resolução em LPO como fechamento e ponte para Prolog.

Fio condutor

Como passar de regras gerais com variáveis para uma conclusão concreta que responda a uma consulta?

Retomada: a LPO nos deu a linguagem

Retomando

Na Parte I, representamos objetos, relações, funções e regras gerais utilizando $\forall$ e $\exists$.

$$\forall x (King(x) \wedge Greedy(x) \Rightarrow Evil(x))$$

King(John) *Greedy(John)*

Pergunta

Como um algoritmo reconhece que a regra geral pode ser aplicada especificamente a *John*?

Por que inferência em LPO exige um passo a mais?

- Na lógica proposicional, símbolos como P e Q já são unidades fechadas.
- Em LPO, fórmulas podem conter variáveis e quantificadores.
- Antes de aplicar certas regras de inferência, precisamos descobrir **quais objetos** tornam expressões compatíveis.

Ideia central

Grande parte da inferência em LPO depende de encontrar e aplicar **substituições** adequadas.

Substituição

Uma substituição θ associa variáveis a termos.

$$\theta = \{x/\text{John}, y/\text{Mary}\}$$

Para $P(x, y)$:

$$SUBST(\theta, P(x, y)) = P(John, Mary)$$

- o termo substituto pode ser uma constante, variável ou termo funcional;
- a substituição deve ser aplicada de forma consistente em toda a expressão.

Instanciação universal - IU

De uma sentença universal podemos obter qualquer instância substituindo a variável por um termo básico.

$$\frac{\forall v \alpha}{SUBST(\{v/g\}, \alpha)}$$

Exemplo:

$$\forall x (King(x) \Rightarrow Person(x))$$

permite obter

$$\text{King}(\text{John}) \Rightarrow \text{Person}(\text{John}).$$

Observação

A IU especializa uma regra geral para um termo do domínio.

Uma possibilidade: proposicionalizar a KB

- 1 usar instanciação para gerar versões sem variáveis das sentenças;
- 2 tratar cada sentença básica como um símbolo proposicional;
- 3 aplicar algoritmos de inferência proposicional.

Exemplo:

$$\forall x (King(x) \wedge Greedy(x) \Rightarrow Evil(x))$$

gera, para *John*:

$$King(John) \wedge Greedy(John) \Rightarrow Evil(John).$$

Por que não proposicionalizar tudo?

- podem ser produzidas muitas instâncias irrelevantes para a consulta;
- símbolos funcionais permitem termos de profundidade crescente:

Father(John), Father(Father(John)), ...

- precisamos de mecanismos que trabalhem diretamente com variáveis e estruturas de termos.

Limite teórico

A consequência lógica em LPO é semi-decidível: quando uma consequência existe, há procedimentos capazes de encontrá-la; em geral, não há garantia de término para todas as respostas negativas.

Modus ponens generalizado - MPG

Ideia

Se uma substituição torna os antecedentes de uma regra compatíveis com fatos da KB, podemos inferir o conseqüente com a mesma substituição.

$$p'_1, \dots, p'_n, (p_1 \wedge \dots \wedge p_n \Rightarrow q)$$

com $SUBST(\theta, p'_i) = SUBST(\theta, p_i)$, então:

$$SUBST(\theta, q)$$

Exemplo de MPG

$$\forall x (King(x) \wedge Greedy(x) \Rightarrow Evil(x))$$

$$King(John) \quad Greedy(John)$$

Uma substituição possível é:

$$\theta = \{x/John\}$$

Logo:

$$Evil(John)$$

Pergunta

O ponto difícil é aplicar o modus ponens ou descobrir a substituição θ ?

Unificação

Definição

Unificar duas expressões significa encontrar uma substituição que as torne idênticas.

$$UNIFY(p, q) = \theta$$

tal que

$$SUBST(\theta, p) = SUBST(\theta, q).$$

Exemplo:

$$UNIFY(Knows(John, x), Knows(John, Jane)) = \{x/Jane\}.$$

Unificação: exemplos

p	q	θ
$Knows(John, x)$	$Knows(John, Jane)$	$\{x/Jane\}$
$Knows(John, x)$	$Knows(y, Bill)$	$\{x/Bill, y/John\}$
$Knows(John, x)$	$Knows(y, Mother(y))$	$\{x/Mother(John), y/John\}$
$Knows(John, x)$	$Knows(x, Elizabeth)$	$false$

Pergunta

Em quais casos a unificação falha? O que há estruturalmente diferente nesses pares?

Unificador mais geral - UMG

Considere:

$Knows(John, x)$ $Knows(y, z)$

Uma unificação é:

$\{y/John, x/z\}$

Outra, mais específica, seria:

$\{y/John, x/John, z/John\}$

Observação

O **unificador mais geral** impõe o mínimo de restrições necessário; outras unificações podem ser obtidas a partir dele.

Padronização de variáveis

Duas regras podem reutilizar nomes de variáveis sem que essas variáveis sejam o mesmo objeto lógico.

Knows(x, Elizabeth)

Knows(John, x)

Antes de unificar, podemos renomear a primeira variável:

Knows(z, Elizabeth)

e então:

$$\theta = \{z/John, x/Elizabeth\}.$$

Observação

Essa operação é chamada de padronização à parte (*standardize apart*).

Quando a unificação falha?

- símbolos de predicado diferentes: $P(x)$ e $Q(x)$;
- constantes incompatíveis: $Knows(John, x)$ e $Knows(Mary, Bob)$ no primeiro argumento;
- estruturas incompatíveis de termos;
- teste de ocorrência (*occurs check*): impedir $x = f(x)$.

Pergunta

Por que permitir $x = f(x)$ criaria uma substituição problemática?

Atividade em classe - unificação

Em duplas - 6 minutos

Encontre um UMG para cada par ou indique por que a unificação falha.

- 1 $P(x, a)$ e $P(b, y)$
- 2 $Knows(John, x)$ e $Knows(y, Mary)$
- 3 $Parent(x, x)$ e $Parent(John, Mary)$
- 4 $P(x)$ e $P(f(x))$

Depois

Qual desses exemplos exige o teste de ocorrência?

Cláusulas definidas

Uma cláusula definida possui exatamente um literal positivo e pode ser escrita como regra:

$$P_1 \wedge P_2 \wedge \cdots \wedge P_n \Rightarrow Q$$

ou como fato, quando não há antecedentes:

$$Q$$

Observação

Essa forma é particularmente conveniente para encadeamento e será importante na transição para Prolog.

Exemplo condutor: o caso West

Considere uma KB com fatos e regras como:

American(West)

Missile(M1)

Owns(Nono, M1)

Sells(West, M1, Nono)

Enemy(Nono, America)

e regras que relacionam mísseis, armas, hostilidade e criminalidade.

Pergunta

Como provar *Criminal(West)* sem gerar todas as consequências possíveis da KB?

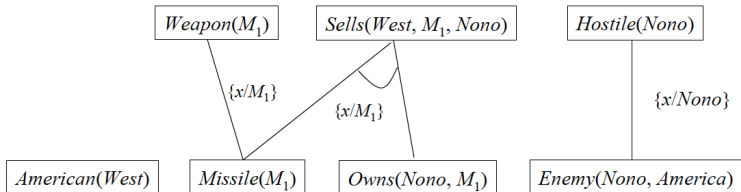
Encadeamento direto - forward chaining

- começa pelos fatos conhecidos;
- procura regras cujos antecedentes possam ser satisfeitos;
- unifica variáveis, gera novas conclusões e as adiciona à KB;
- repete até obter a consulta ou atingir um ponto fixo.

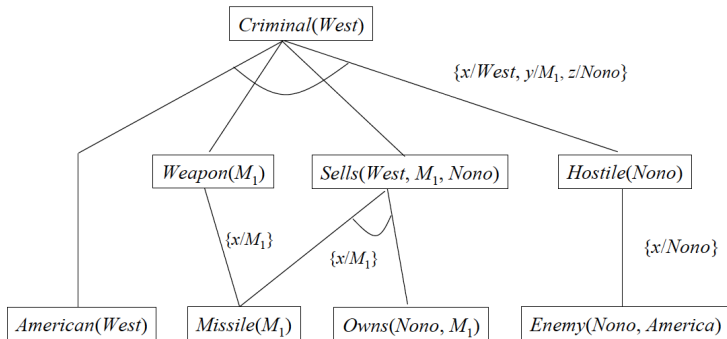
Observação

É uma estratégia orientada pelos dados (*data-driven*).

Encadeamento direto no caso West



Árvore de derivação - encadeamento direto



Pergunta

Quais fatos precisaram ser derivados antes de *Criminal(West)* aparecer?

Encadeamento direto: visão do algoritmo

- 1 selecionar uma regra ainda aplicável;
- 2 padronizar suas variáveis;
- 3 encontrar substituições que satisfaçam todos os antecedentes;
- 4 aplicar a substituição ao consequente;
- 5 inserir uma conclusão que ainda não esteja na KB;
- 6 parar quando a consulta for obtida ou nada novo puder ser produzido.

Observação

Para cláusulas definidas, o encadeamento direto é correto e completo para as consequências expressáveis nesse fragmento.

Quando o encadeamento direto é adequado?

- há muitos fatos chegando ao sistema;
- queremos manter consequências atualizadas;
- há várias consultas possíveis;
- o domínio possui regras acionadas naturalmente pelos dados.

Discussão

Qual é o custo de derivar fatos que nunca serão relevantes para a consulta atual?

Encadeamento reverso - backward chaining

- começa pela consulta que desejamos provar;
- procura fatos ou regras cujo conseqüente possa unificar com a meta;
- transforma os antecedentes dessas regras em novas submetas;
- continua até encontrar fatos que sustentem todas as submetas.

Observação

É uma estratégia orientada pela meta (*goal-driven*).

Encadeamento reverso no caso West

Meta inicial:

Criminal(West)

Uma regra compatível reduz a prova às submetas:

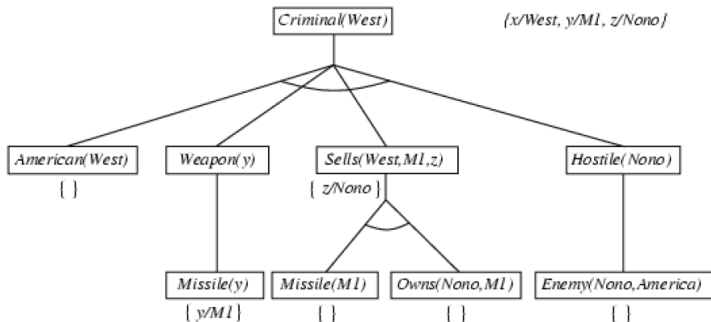
American(West), Weapon(y), Sells(West, y, z), Hostile(z)

Cada submeta é então resolvida por fatos ou por novas regras.

Pergunta

Qual é a vantagem de iniciar pela meta em vez de derivar todas as consequências da KB?

Árvore de prova - encadeamento reverso



Alternativas e retrocesso - backtracking

Uma submeta pode unificar com mais de uma regra ou fato.

- escolhemos uma alternativa;
- tentamos provar suas submetas;
- se a tentativa falhar, voltamos ao último ponto de escolha;
- tentamos outra alternativa.

Conexão com busca

O mecanismo se parece com uma busca em profundidade sobre alternativas de prova.

Encadeamento direto versus reverso

	Direto	Reverso
Orientação	dados	meta
Início	fatos conhecidos	consulta
Expansão	regras aplicáveis	regras que podem provar a meta
Risco	derivar fatos irrelevantes	repetir/subexplorar submetas
Uso típico	regras reativas, múltiplas consultas	consultas específicas, Prolog

Resolução em LPO: visão geral

A resolução oferece um procedimento uniforme de prova por refutação.

- 1 adicionar a negação da consulta à KB;
- 2 converter as sentenças para forma normal conjuntiva (FNC);
- 3 padronizar variáveis e unificar literais complementares;
- 4 aplicar resolução;
- 5 derivar a cláusula vazia para demonstrar contradição.

Observação

Nesta aula, a resolução em LPO é uma introdução: o foco principal está em unificação e encadeamento.

Conversão para FNC em LPO

Roteiro conceitual:

- 1 eliminar $\Leftrightarrow$ e $\Rightarrow$;
- 2 mover negações para dentro;
- 3 padronizar variáveis;
- 4 eliminar quantificadores existenciais por Skolemização;
- 5 eliminar quantificadores universais explícitos;
- 6 distribuir $\vee$ sobre $\wedge$;
- 7 separar em cláusulas.

Resolução em LPO: unificar antes de cancelar

Considere:

$$\neg \text{Animal}(F(x)) \vee \text{Loves}(G(x), x)$$

$$\neg \text{Loves}(u, v) \vee \text{Kills}(u, v)$$

Para resolver, precisamos unificar:

$$\text{Loves}(G(x), x) \quad \text{e} \quad \text{Loves}(u, v)$$

com

$$\theta = \{u/G(x), v/x\}.$$

O resolvente é:

$$\neg \text{Animal}(F(x)) \vee \text{Kills}(G(x), x).$$

Prova por refutação - ideia mínima

Suponha que desejamos provar $Mortal(Socrates)$ a partir de:

$$\forall x (Human(x) \Rightarrow Mortal(x))$$

$$Human(Socrates)$$

Adicionamos:

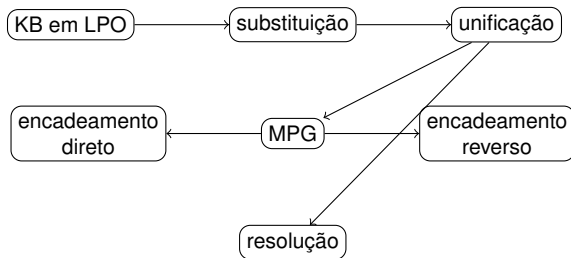
$$\neg Mortal(Socrates)$$

A FNC contém:

$$\neg Human(x) \vee Mortal(x), \quad Human(Socrates), \quad \neg Mortal(Socrates).$$

Unificando $x/Socrates$ e resolvendo, obtemos a cláusula vazia $\square$.

Mapa conceitual da inferência em LPO



Observação

A unificação é o mecanismo que reaparece em vários procedimentos de inferência em LPO.

Atividade de fechamento - qual estratégia usar?

Em duplas ou trios - 7 minutos

Para cada cenário, escolha encadeamento direto ou reverso e justifique.

- 1 Um sistema recebe continuamente fatos de sensores e deve disparar alertas.
- 2 Um usuário pergunta se um indivíduo específico satisfaz uma propriedade.
- 3 Há milhares de fatos, mas somente uma consulta pontual.
- 4 Queremos materializar previamente consequências para diversas consultas futuras.

Conexão

Qual dessas estratégias se aproxima mais do funcionamento declarativo que veremos em Prolog?

Síntese da Parte II

- instanciação transforma sentenças quantificadas em casos utilizáveis;
- substituição e unificação permitem compatibilizar expressões com variáveis;
- o MPG aplica regras gerais a fatos compatíveis;
- encadeamento direto parte dos dados; encadeamento reverso parte da meta;
- resolução combina FNC, unificação e prova por refutação.

Cheque sua compreensão

Por que a unificação é mais do que uma simples substituição de texto?

Próxima aula: Prolog

E se fatos, regras, unificação, encadeamento reverso e retrocesso fizerem parte diretamente do modelo de execução de uma linguagem?

Aula 07

Prolog: fatos, regras, consultas, unificação, resolução de metas e retrocesso.

Dúvidas



Dúvida é o começo da sabedoria.